Final Project Writeup

Cricket Bergner

05/04/26

1 Introduction

The Final Project explores the dynamics of the magnetic pendulum through numerical simulation, visualization, and analysis of chaotic behavior. For multiple fixed magnets, the motion of the magnetic pendulum is simulated using a system of differential equations that account for restoring forces, damping, and magnetic attraction, solved by multiple numerical integrators. Trajectory plots show how different initial conditions evolve over time. The fractals generated show how the slightest change in initial condition can influence what magnet the pendulum will settle on. Animations are shown to model how different integration methods with close initial conditions lead to divergent outcomes in chaotic systems. Conceptually, these components emphasize the physical behavior of non-linear dynamical systems and the impact of numerical methods on predicting long term behavior.

1.1 Languages and Libraries

The code for this project was written in Python. The libraries used were numpy, matplotlib.pyplot, scipy.integrate.solve_ivp, random, and matplotlib.colors.ListedColorMap, and matplotlib.animation.

This was the first project to use the ListedColorMap library. The library is used to assign distinct colors to different magnets in the fractal points. This helps with visualizing the basins of attraction generated by each magnet.

The solve_ivp library was brought in again to assist in numerically solving the differential equations that mapped the magnetic pendulum's motion. It was able to solve for the RK45 and DOP853 integrators, which was useful when doing integrator comparisons.

Finally, the random library was a big help in showing the sensitivity of the system to initial conditions.

2 Undergirding Theory

2.1 The Magnetic Pendulum

The magnetic pendulum is a system of deterministic chaos, meaning it will change in the same way from the same starting conditions. It is set up by placing multiple magnets, north

pole face up, on a plane. At the end of a pendulum is another magnet, south pole side down. The pendulum's magnet will be attracted to the magnets on the table and will eventually come to rest over one of them. The time and path it takes to do this is heavily determined by initial conditions (x_0, y_0, v_{x_0}, and v_{y_0}), magnetic strength, the damping force (usually friction, given by R), the restoring force (the pendulum will want to go back to resting in the center of the grid, given by C), and the distance (d) between the pendulum bob and the plane of magnets. The motion that the pendulum will follow based on these conditions is given by the equations below.

$$x'' + Rx' - \sum_{i=1}^{3} \frac{x_i - x}{\left(\sqrt{(x_1 - x)^2 + (y_1 - y)^2 + d^2}\right)^3} + Cx = 0$$

$$y'' + Ry' - \sum_{i=1}^{3} \frac{y_i - y}{\left(\sqrt{(x_1 - x)^2 + (y_1 - y)^2 + d^2}\right)^3} + Cy = 0$$

Suppose you had an N by N grid of initial x and y points for the magnetic pendulum system. If you carefully went through each point, and mapped which magnet the pendulum would swing to, you would get an interesting map that shows the deterministic chaos of the system. This map is a fractal.

2.2 Fractals

A fractal is a geometric shape that has complex detail at large and small scales, and often shows self similarity, meaning smaller parts resemble the whole. In the magnetic pendulum, the boundary between regions that fall into different magnets becomes extremely intricate the further away from the magnet the pendulum is started. Even when zoomed in, the patterns remain. The motion represents a fractal as the tiny changes in position lead to entirely different outcomes, creating infinitely detailed boundaries between the basins of attraction, which is an indication of a chaotic system.

2.3 Numerical Integration Methods

Numerical integration methods are used to approximate the solutions to systems of ordinary differential equations. These methods advance the solution forward in time by estimating how the system evolves over small time steps. The accuracy and computational efficiency of each method depends on how well it approximates the derivative in each step, as well as how well it controls numerical error. Three different methods were used in this project.

2.3.1 Runge Kutta Fehlburg (RK45)

RK45 is an adaptive Runge–Kutta method that computes two approximations of different orders (fourth and fifth) at each step. The difference between these two estimates provides an approximation of the local truncation error, which is then used to adjust the step size dynamically. If the error is too large, like in regions where the function is more complex, the

step size is shrunk. If the error is small, like in smooth curves or linear slopes, larger steps can be taken.

The global error (error over time) of RK45 scales as $O(h^5)$. Due to the adaptive step size, this method is efficient for a wide range of problems with varying time scales. However, since each step requires multiple function evaluations, this does increase the computational cost per step as opposed to simpler methods. Despite its accuracy, RK45 is not symplectic, meaning it does not conserve energy in Hamiltonian systems and may exhibit long-term drift in conserved quantities.

2.3.2 Non-Iterative Heun's Method

The non-iterative Heun's Method improves upon the basic Euler method by incorporating information about the slope at both the beginning and the end of each time step. It then averages the initial and final slopes together before it takes the next step. This produces a more accurate estimate of the system's evolution over the step without requiring iterative refinement.

The global error of Heun's method scales as $O(h^2)$. This means that reducing the step size by a factor of two reduces the error by approximately a factor of four. This method uses a fixed step size, which prevents it from adapting to regions of the function where the solution changes rapidly, which can lead to a loss of accuracy in non-linear systems. While computationally inexpensive, the boon of a faster runtime comes with lower accuracy when compared to higher order methods. As a result, to achieve reliable results, the number of total steps needs to be increased. Heun's Method is not symplectic.

2.3.3 DOP853

DOP853 (Runge-Kutta due to Dormand and Prince) is a high-order Runge–Kutta method of order eight, designed for high precision integration of smooth systems. Like RK45, it uses embedded lower-order estimates to control error and adapt the step size dynamically. DOP853 achieves its high accuracy by sampling the system at many intermediate points within each step and combining these evaluations in a carefully weighted manner.

The global error of DOP853 scales as $O(h^8)$, which allows it to obtain high accuracy even with large step sizes. Due to the increased number of function evaluations per step to achieve such a small truncation error, DOP853 is computationally more expensive to run than the other two methods listed. Thus, its efficiency depends on the balance between fewer steps and higher per-step cost. Like the other two methods, DOP853 is not symplectic and may accumulate energy error over long simulations.

3 Walking Through the Code

Before the code, all relevant libraries were imported. The following code was split into a header file and the code file, which was divided further into three sections for logical clarity. The contents of the header file will be explained first, with the main body of the code explained afterwards.

```python
# import libraries
import numpy as np
from matplotlib import pyplot as plt
from scipy.integrate import solve_ivp as intg
import random as ra
from matplotlib.colors import ListedColormap # for the fractal mapping
from matplotlib import animation
from FinalProject_Header import *
```

3.1 Functions

3.1.1 Acceleration

This function computes the net force per unit mass on the pendulum by combining the damping force, restoring force, and the attraction from each magnet (magnetic strength). It loops over the magnets to find their force contributions to the acceleration in the x and y directions, then returns these values.

```python
def acceleration(x, y, vx, vy, magnets):
    ax = -restoring * x - damping * vx
    ay = -restoring * y - damping * vy

    for mag in magnets:
        dx = mag[0] - x
        dy = mag[1] - y
        r = np.sqrt(dx**2 + dy**2 + vertical_offset**2)
        ax += magnetic_strength * dx / r**3
        ay += magnetic_strength * dy / r**3

    return ax, ay
```

3.1.2 Equations

This function converts the second-order motion into a system of first-order differential equations for use with the ODE solver. Returns velocity components and acceleration components so the solver can evolve position and velocity over time. Since it directly feeds into the integrator, its smoothness and correctness are crucial.

```python
def equations(t, state, magnets):
    x, y, vx, vy = state
    ax, ay = acceleration(x, y, vx, vy, magnets)
    return [vx, vy, ax, ay]
```

3.1.3 RK45

RK45 numerically solves the system of differential equations using an adaptive Runge–Kutta Fehlburg method. It estimates the solution at two different orders to control error automatically. Since it is not symplectic, it does not conserve energy and may drift over longer

simulations. The tolerances are set low to balance fractal structure and computation time. The function will integrate the motion from the initial state until either the time limit is reached or the stop event is triggered.

```
def rk45(initial_state, magnets):
    solution = intg(
        fun = lambda t, y: equations(t, y, magnets),
        t_span=t_span,
        y0=initial_state,
        method='RK45',
        rtol=1e-5,
        atol=1e-7,
        events=lambda t, y: stop_event(t, y, magnets)
    )
    return solution
```

3.1.4 Find_Closest_Magnet

This function determines which magnet a point is closest to and classifies the final state into a basin of attraction. Conceptually, it has a sensitivity to small position differences at the edges of the fractal grid, as tiny integration errors from the integrators can flip which magnet is chosen, which can slightly alter the resulting fractal pattern. Thus, the more precise the integrator, the more efficient this function will be.

```
def find_closest_magnet(x, y, magnets):
    distances = [np.sqrt((x - m[0])**2 + (y - m[1])**2)
                 for m in magnets]
    return np.argmin(distances)
```

3.1.5 Generate_Fast_Fractal

This function simulates the motion of many initial conditions simultaneously using vectorized operations instead of individual ODE solvers. It evolves all points forward in time, tracks how quickly they settle, and assigns each point to a magnet to build the fractal image. The result is stored as an array.

On the conceptual side, this function trades accuracy for speed. Instead of solving each trajectory carefully, it approximates all of them quickly to reveal the global structure. Euler's method has an error of $O(h)$, making it less accurate than the integration functions used. However, since the fractals depend on qualitative behavior rather than precise trajectories, this lower accuracy is often sufficient to capture the larger picture of the fractal and its basins of attraction. However, it is important to keep the step size small to avoid distortion.

```
def generate_fast_fractal(magnets, res=400, max_iter=400):

    # vectorized grid instead of nested loops
    x = np.linspace(-1.5, 1.5, res)
```

```
    y = np.linspace(-1.5, 1.5, res)
    X, Y = np.meshgrid(x, y)

    position = np.stack([X, Y], axis=-1)
    vel = np.zeros_like(position)
    dt = 0.05
    settled_at = np.zeros((res, res))

    for i in range(max_iter):
        accel = -restoring * position - damping * vel

        for m in magnets:
            diff = m - position
            r = np.sqrt(np.sum(diff**2, axis=-1) + vertical_offset**2)
            accel += magnetic_strength * diff / r[..., np.newaxis]**3

        # fast integration (Euler instead of RK45)
        vel += accel * dt
        position += vel * dt
        speed = np.sqrt(np.sum(vel**2, axis=-1))
        settled_at[speed < 0.02] = i

    # determine basins
    distances = []
    for m in magnets:
        dist = np.sqrt(np.sum((position - m)**2, axis=-1))
        distances.append(dist)

    basins = np.argmin(distances, axis=0)

    return basins, settled_at
```

3.1.6 Stop_Event

This function defines a condition to stop integration when the pendulum's speed drops below a threshold. This improves computational efficiency. If the threshold is too large, trajectories may terminate prematurely. If the threshold is too small, computation time is wasted. Choosing a reasonable cutoff is important for balancing accuracy and efficiency.

```
def stop_event(t, state, magnets):
    vx = state[2]
    vy = state[3]
    speed = np.sqrt(vx**2 + vy**2)
    return speed - 0.01

stop_event.terminal = True
```

```
stop_event.direction = -1
```

3.1.7 Generate_Polygon_Magnets

This function generates evenly spaced magnet positions around a circle of radius one for any number of magnets.

```
def generate_polygon_magnets(N, R=1.0):
    magnets = [
        np.array([
            R * np.cos(2*np.pi*k/N),
            R * np.sin(2*np.pi*k/N)
        ])
        for k in range(N)
    ]
    return magnets
```

3.1.8 Plot_Random_Trajectories

Plot_Random_Trajectories randomly generates initial position and velocity in the x and y direction, then visualizes their trajectories using the RK45 method. Conceptually, it gives a qualitative view of how the pendulum moves and how sensitive it is to starting conditions. The accuracy of the plotted paths depends on RK45's tolerances and adaptive stepping. Since trajectories in chaotic systems diverge rapidly, even small numerical differences can lead to visibly different paths over time.

This function also utilizes subplots to map the three random motion plots together for visual clarity.

```
def plot_random_trajectories(magnets, title):

    initial_conditions, solutions = [], []

    # generate random initial conditions
    for _ in range(3):
        x0 = ra.uniform(-1.5, 1.5)
        y0 = ra.uniform(-1.5, 1.5)
        vx0 = ra.uniform(-0.1, 0.1)
        vy0 = ra.uniform(-0.1, 0.1)
        initial_conditions.append([x0, y0, vx0, vy0])

    # solve trajectories
    for ic in initial_conditions:
        solutions.append(rk45(ic, magnets))

    # plot
    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
```

```
    for i, sol in enumerate(solutions):
        ax = axes[i]
        x = sol.y[0]
        y = sol.y[1]

        ax.plot(x, y, lw=0.7)

        for m in magnets:
            ax.scatter(m[0], m[1], color='red',
                       s=50, zorder=3)

        ic = initial_conditions[i]
        ax.set_title(
            f"x0={ic[0]:.2f}, y0={ic[1]:.2f}\n"
            f"vx0={ic[2]:.2f}, vy0={ic[3]:.2f}"
        )

        ax.set_aspect('equal')
        ax.grid(alpha=0.3)
        ax.set_xlim(-1.5, 1.5)
        ax.set_ylim(-1.5, 1.5)

    plt.suptitle(title)
```

3.1.9 Heun_Non_Iterative

Heun's Method (Non-Iterative) implements a fixed-step, second-order method that averages slopes at the beginning and end of each step. It is an improvement on Euler's Method as it accounts for how the slope will change over a step. This is a fixed step process, so at a larger number of steps it will run slower. However, the larger number of steps will also improve accuracy under more complex systems. This function also calls on the stopping condition to reduce runtime.

```
def heun_non_iterative(initial_state, magnets, dt, steps):

    state = np.array(initial_state, dtype=float)
    trajectory = [state.copy()]

    for _ in range(steps):
        k1 = np.array(equations(0, state, magnets))
        predictor = state + dt * k1
        k2 = np.array(equations(0, predictor, magnets))

        state = state + (dt/2) * (k1 + k2)
        trajectory.append(state.copy())
```

```
        # stopping condition (velocity gets too slow)
        speed = np.sqrt(state[2]**2 + state[3]**2)
        if speed < 0.01:
            break

    return np.array(trajectory)
```

3.1.10 DOP853

This function uses a high-order (8th order) Runge–Kutta method with adaptive step sizing for very accurate solutions. It is able to pull from the solve_ivp library to help build its solution. Conceptually, it samples the system many times within each step to build a highly precise estimate of the trajectory. Due to this, DOP853 can achieve high accuracy even with relatively large steps. However, all the samples make it more computationally expensive. Since it is also not symplectic, the long term energy behavior may still drift.

```
def dop853(initial_state, magnets):
    solution = intg(
        fun=lambda t, y: equations(t, y, magnets),
        t_span=t_span,
        y0=initial_state,
        method='DOP853',
        rtol=1e-7,
        atol=1e-9,
        events=lambda t, y: stop_event(t, y, magnets)
    )
    return solution
```

3.1.11 Pad_Trajectories

Extends shorter trajectories so all the integrator outputs have the same length for comparison and animation. This is to allow for the integrators that take longer to estimate the path of motion to not be cut off prematurely, which is essential for creating a clean visual for the GIF file.

```
def pad_trajectory(traj, target_len):
        current_len = len(traj)
        if current_len < target_len:
            padding = np.tile(traj[-1], (target_len - current_len, 1))
            return np.vstack((traj, padding))
        return traj
```

3.1.12 Animate_Integrators

This function generates side-by-side trajectory animations to compare RK45, Heun's Method, and DOP853. This is done to highlight how different numerical methods diverge over time,

especially in a chaotic system. Each system is given a different, but very similar, set of initial points to emphasize how in an increasingly chaotic system, even the smallest difference can result in significantly different outcomes. Since each method has different accuracy and step behavior, differences grow over time and are expected. The pad_trajectories function is utilized here to make sure the animation captures the complete trajectory over time of each integrator. Different colors and labels are given to the different integrators to make them visually distinct.

```
def animate_integrators(magnets, initial_states, steps, filename_prefix):
    dt = (t_span[1] - t_span[0]) / steps

    # generate trajectories with slightly different start points
    rk = rk45(initial_states[0], magnets)
    dp = dop853(initial_states[1], magnets)
    hn = heun_non_iterative(initial_states[2], magnets, dt, steps)

    rk_xy = np.vstack((rk.y[0], rk.y[1])).T
    dp_xy = np.vstack((dp.y[0], dp.y[1])).T
    hn_xy = hn[:, :2]

    # making sure the animations run for all the integrators,
    # even the slower ones.
    max_len = max(len(rk_xy), len(dp_xy), len(hn_xy))

    rk_xy = pad_trajectory(rk_xy, max_len)
    dp_xy = pad_trajectory(dp_xy, max_len)
    hn_xy = pad_trajectory(hn_xy, max_len)

    # plot
    fig, ax = plt.subplots(figsize=(7, 7))
    ax.set_xlim(-2.0, 2.0)
    ax.set_ylim(-2.0, 2.0)
    ax.set_aspect('equal')
    ax.set_title(f"{len(magnets)} Magnets ({steps} steps)")

    for i, m in enumerate(magnets):
        ax.scatter(m[0], m[1], color='black', s=50, zorder=3)

    rk_line, = ax.plot([], [], color='#71e669', label='RK45 (y0=1.20)')
    dp_line, = ax.plot([], [], color='blue', label='DOP853 (y0=1.20)')
    hn_line, = ax.plot([], [], color='red', label='Heun (y0=1.20)')
    ax.legend(loc='upper right')
```

3.1.13 Update

Updates frames in the animation with the functions' different positions. It also tracks how far each point is away from the other integrators throughout the frames, to show how different factors such as increased step size could improve accuracy for certain methods. The frames are compiled together in a GIF file, and the table of distances is printed to the terminal.

```
def update(frame):
      rk_line.set_data(rk_xy[:frame, 0], rk_xy[:frame, 1])
      dp_line.set_data(dp_xy[:frame, 0], dp_xy[:frame, 1])
      hn_line.set_data(hn_xy[:frame, 0], hn_xy[:frame, 1])
      return rk_line, dp_line, hn_line

  ani = animation.FuncAnimation(fig, update, frames=max_len - 1,
          interval=50, blit=True)
  ani.save(f"{filename_prefix}.gif", writer='pillow')
  plt.close()

  # printing the table of distances
  f = max_len - 1
  d_rk_dp = np.linalg.norm(rk_xy[f] - dp_xy[f])
  d_rk_hn = np.linalg.norm(rk_xy[f] - hn_xy[f])
  d_hn_dp = np.linalg.norm(hn_xy[f] - dp_xy[f])

  print("")
  print("-" * 40)
  print(f"--- {filename_prefix} ---")
  print(f"Final distance between RK45 and DOP853: {d_rk_dp:.6f}")
  print(f"Final distance between RK45 and Heun:   {d_rk_hn:.6f}")
  print(f"Final distance between Heun and DOP853: {d_hn_dp:.6f}")
  print("-" * 40)
```

3.2 Motion Plots

The main file starts by defining the initial conditions. The damping and vertical offset were kept to a smaller number to emphasize magnetic interactions in the motion.

```
# initial conditions
damping = 0.2
vertical_offset = 0.3
t_span = (0,20)
initial_conditions, solutions = [], []
```

Multiple magnet configurations are defined and looped over. For each iteration of the loop, the geometry of the magnets is called in, then the random trajectories are plotted. Note that the restoring force is set to zero, as for larger magnets it would lessen the effects of

the magnets on the pendulum motion in certain geometries. The magnetic strength is also increased upon hitting the sixth magnet to show a more dramatic effect in the motion plots.

It is noteworthy that the motion shown below is generated by only the RK45 method, as the purpose was just to show motion around the magnets. The complexities introduced by different integrators will be explored later.

```
# loop over multiple magnet configurations
magnet_counts = [3, 4, 5, 6, 7, 8]

print("")
print("Generating motion plots...")

for idx, N in enumerate(magnet_counts):
    magnets = generate_polygon_magnets(N)

    restoring = 0.0
    magnetic_strength = 1.2 if N <= 5 else 2.0

    # plot the trajectories for N magnets
    plot_random_trajectories(magnets, f"Trajectories ({N} Magnets)")
    plt.savefig(f"trajectories_{N}_magnets.png",
                bbox_inches='tight', dpi=300)
    plt.show()
```

The plots from the code are shown below.

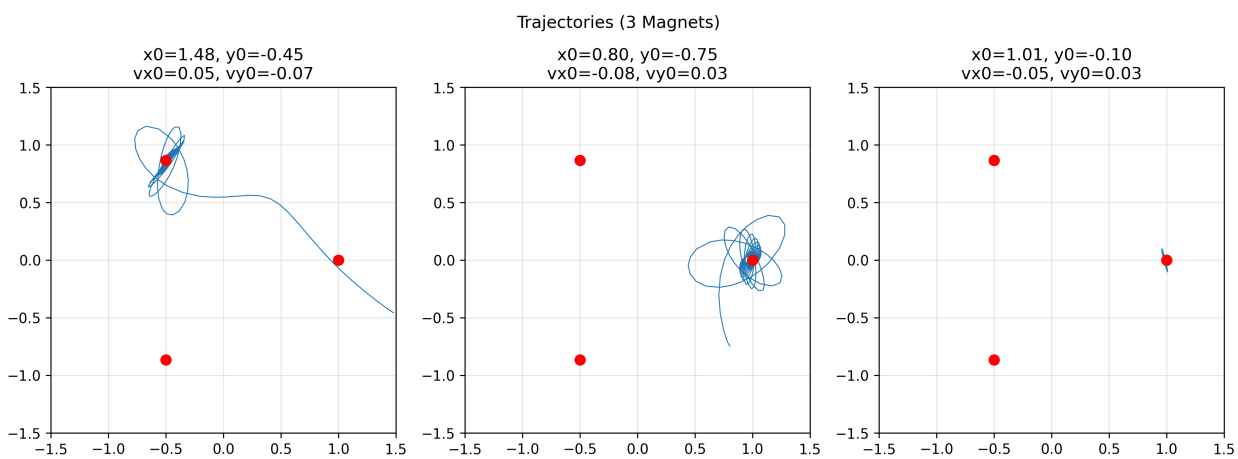

Figure 1: Three different plots of the three magnet configuration, each with a different set of initial variables. Notice that the further the pendulum starts away from the magnets, the more complex and interesting its path of motion is.

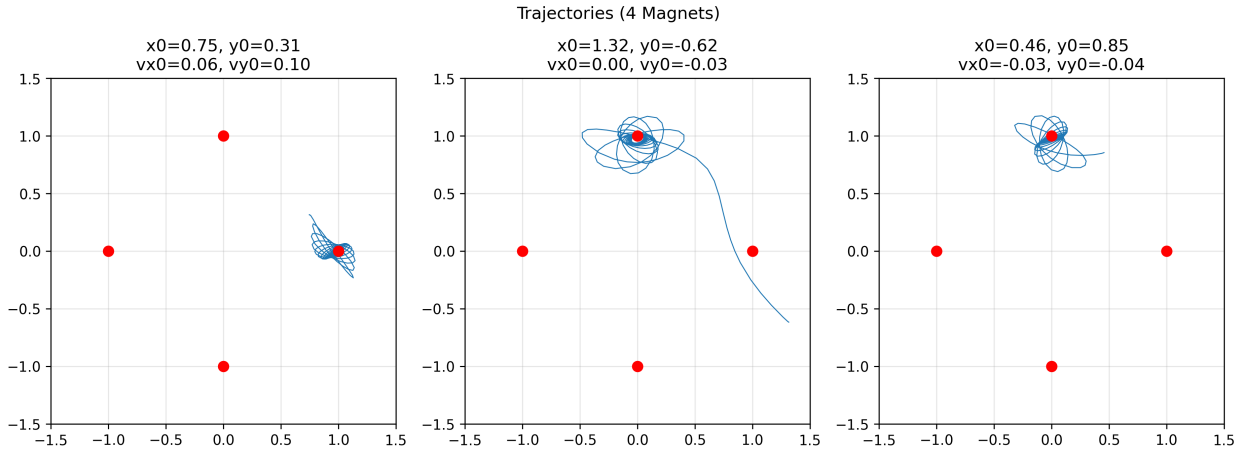

Figure 2: Three different plots of the four magnet configuration, each with a different set of initial variables. Notice that the further the pendulum starts away from the magnets, the more complex and interesting its path of motion is.

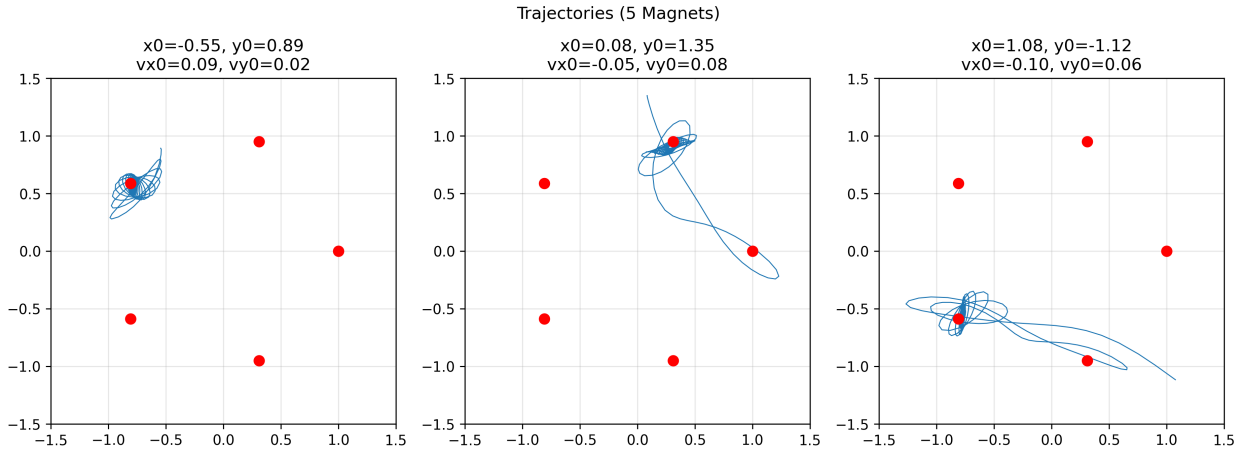

Figure 3: Three different plots of the five magnet configuration, each with a different set of initial variables. Notice that the further the pendulum starts away from the magnets, the more complex and interesting its path of motion is. Another interesting feature is the pendulum's path of swinging around other magnets, as it has not reached a low enough momentum to stop on them.

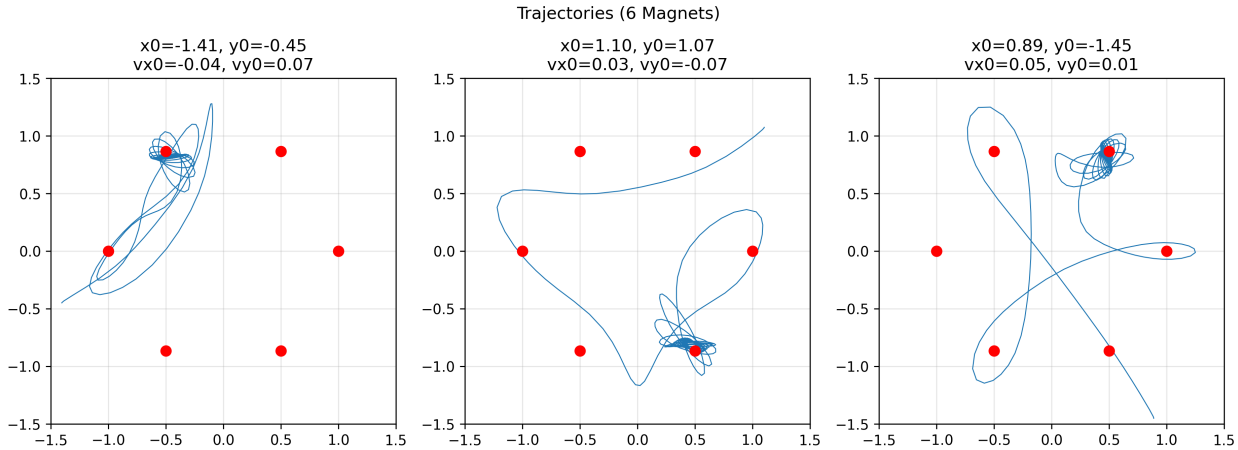

Figure 4: Three different plots of the six magnet configuration, each with a different set of initial variables.

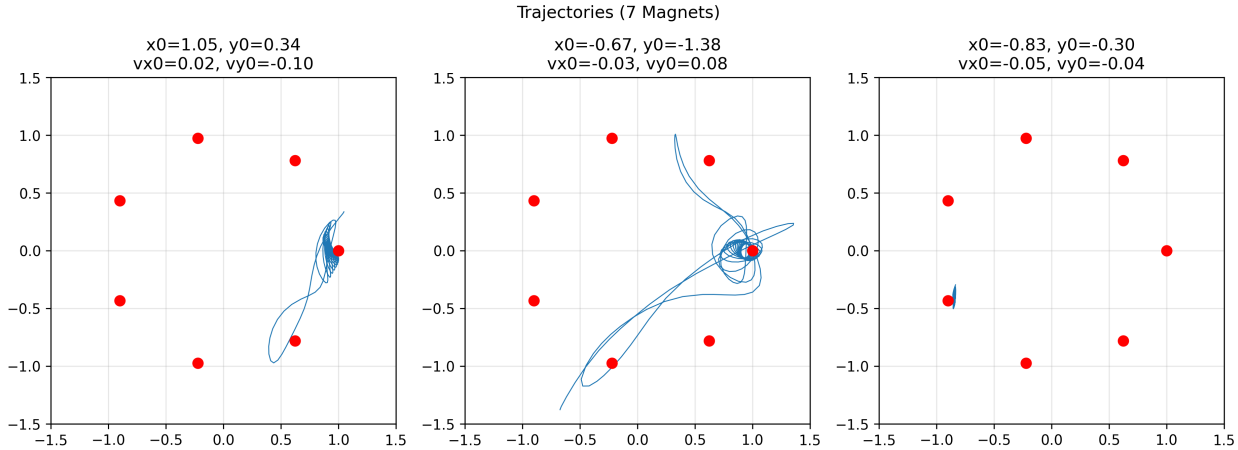

Figure 5: Three different plots of the seven magnet configuration, each with a different set of initial variables.

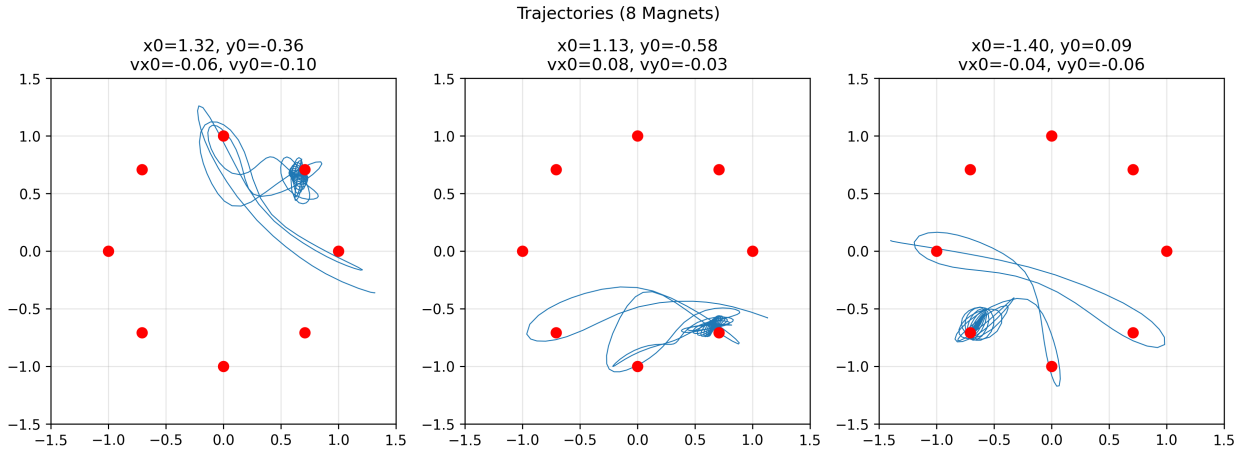

Figure 6: Three different plots of the eight magnet configuration, each with a different set of initial variables.

3.3 Fractals

This section of the code generates the fractals associated with the different magnet configurations. For each magnet value given, magnet positions are arranged symmetrically, and the generate_fast_fractal function is called. Euler's Method is used in this section rather than RK45 to preserve a shorter runtime while still capturing the general shape of the fractal.

The results are displayed as a color-coded image where each color represents a different basin of attraction, with an additional grayscale overlay indicating how quickly each point settles. The magnets themselves are plotted on top of the fractal to provide spatial reference. Each subplot corresponds to a different number of magnets. Finally, all six fractal plots are arranged into a grid, labeled, and saved as a high-resolution figure for analysis and comparison.

```
print("")
print("Generating fractals...")
fig, axes = plt.subplots(2, 3, figsize=(15, 10))  # 6 plots total
axes_nn = axes.flatten()
for idx, N in enumerate(magnet_counts):

    ax = axes_nn[idx]
    magnets = generate_polygon_magnets(N)
    restoring = 0.0
    magnetic_strength = 1.2 if N <= 5 else 2.0

    # generate fractal
    basins, shading = generate_fast_fractal(magnets, res=400)

    # color pallette
    base_colors = [
```

```
        '#582c83', '#FFD100', '#0fc7f2', "#49f866",
        '#fb0cce', '#3e5c1f', '#3e5cf6', '#fe5c1f',
        '#fc0000', '#fabebe', '#008080', '#e6beff',
    ]

    colors = base_colors[:N]  # take first N colors
    custom_cmap = ListedColormap(colors)

    # plot
    ax.imshow(basins, cmap=custom_cmap,
              extent=[-1.5, 1.5, -1.5, 1.5],
              origin='lower',
              vmin=0, vmax=N-1,
              interpolation='nearest',
              aspect='equal')

    ax.imshow(shading, cmap='bone',
              extent=[-1.5, 1.5, -1.5, 1.5],
              origin='lower', alpha=0.25,
              aspect='equal')

    # plot magnets
    for m in magnets:
        ax.scatter(m[0], m[1], color='white', s=20)

    ax.set_title(f"{N} Magnets")
    ax.set_xticks([])
    ax.set_yticks([])

plt.suptitle("Magnetic Pendulum Fractals (
                Various Magnet Configurations)", fontsize=16)
plt.savefig("magnetic_pendulum_fractals_grid.png",
                bbox_inches='tight', dpi=300)
plt.show()
```

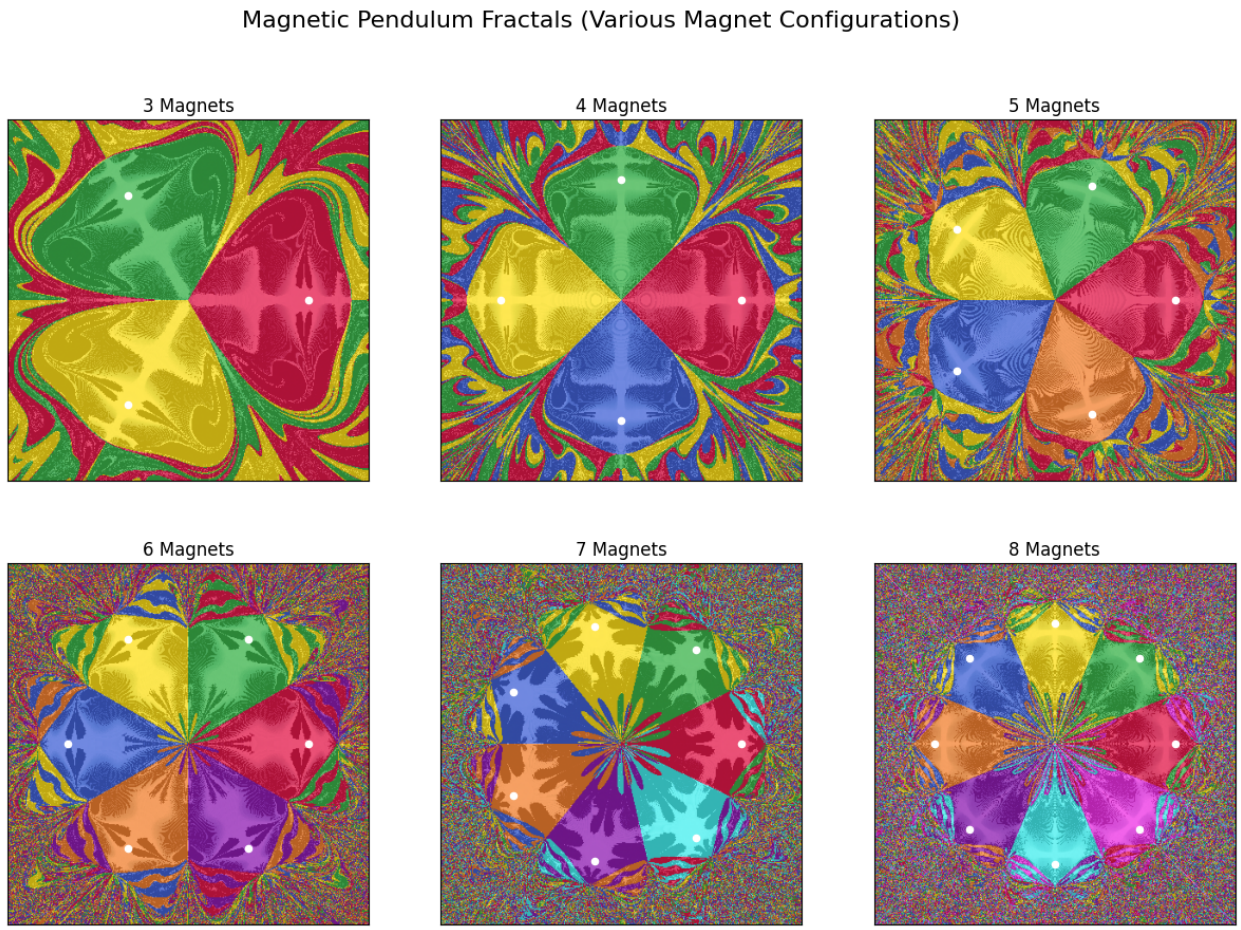

Figure 7: The fractals for three, four, five, six, seven, and eight magnet configurations. Notice how as the number of magnets increase, the noisier the edges of each fractal grid become. This is showing how as more magnets are introduced into the system, the more small differences in initial conditions will heavily influence what magnet will be landed on.

3.4 Animations

The previous sections of the code provide a good basis for the deterministic chaos of the magnetic pendulum system. Now the code will move to exploring how this chaos is mapped by different integrators.

The RK45, Heun's Method, and DOP853 integrators are given the same initial conditions. For three, four, and five magnets, the code plots the paths generated by each respective integrator as they each solve the differential equation given for the motion of the magnetic pendulum. The diverging paths are captured into a GIF file to show the differences in each integrator over time. Additionally, the final distances of each integrator from each other are printed to the screen to show how much divergence occurred. This process is repeated for 512 steps and 2048 steps, to answer the question of whether an increased number of steps leads to a more accurate solution.

```
configs = [3, 4, 5] # number of magnets
```

```
step_sizes = [512, 2048] # to show how step sizes effects the accuracy

for N in configs:
    magnets = generate_polygon_magnets(N)
    restoring = 0.05
    magnetic_strength = 1.2

    # the initial conditions are the same, but could be changed for other simulations
    states = [
        [-1.2, 1.20, 0.0, 0.0], # RK45
        [-1.2, 1.20, 0.0, 0.0], # DOP853
        [-1.2, 1.20, 0.0, 0.0]  # Heun
    ]

    for steps in step_sizes:
        filename = f"{N}_magnets_{steps}_steps"
        animate_integrators(magnets, states, steps, filename)
```

The end frames of each animation and the table of distance values is shown below.

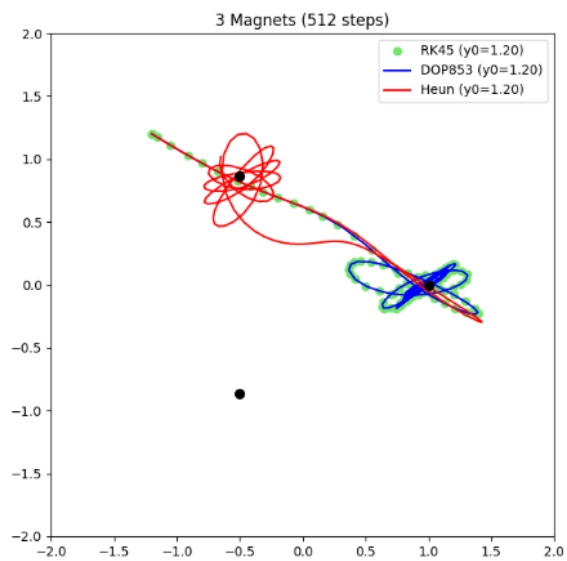

Figure 8: RK45, Heun's Method, and DOP853 integrators comparison for 512 steps and a system of three magnets with the same initial conditions.The non-adaptive step size for Heun's Method shows in its less approximate estimate of the path of motion as compared to the other two integrators.

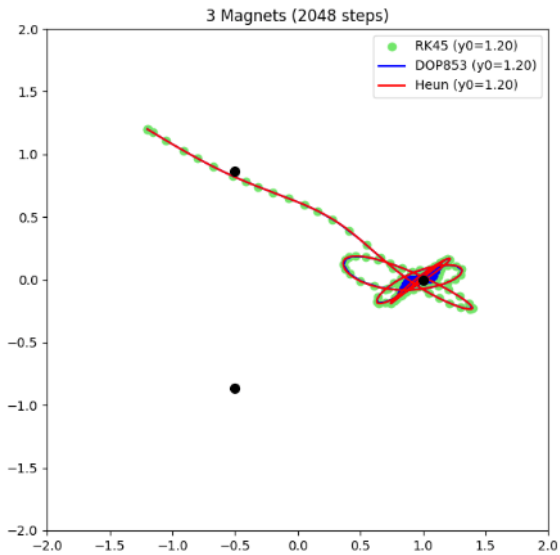

Figure 9: RK45, Heun's Method, and DOP853 integrators comparison for 512 steps and a system of three magnets with the same initial conditions. Despite having a non-adaptive step size, Heun's Method gives a much better approximation of the motion when it has an increased number of steps.

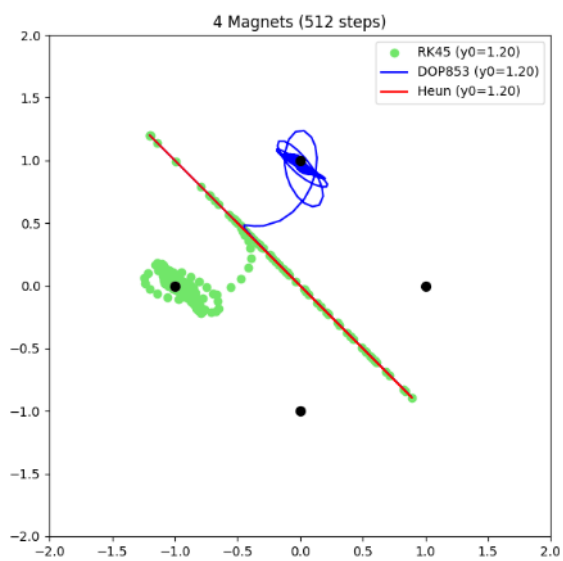

Figure 10: RK45, Heun's Method, and DOP853 integrators comparison for 512 steps and a system of four magnets. Despite having the same starting conditions, RK45 and DOP853 vary in their prediction for the motion. The plot for 2048 steps is identical and thus not included in the writeup.

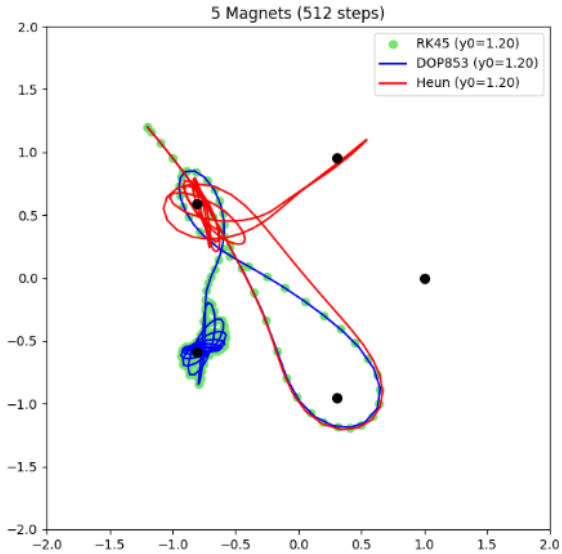

Figure 11: RK45, Heun's Method, and DOP853 integrators comparison for 512 steps and a system of five magnets with the same initial conditions.The non-adaptive step size for Heun's Method shows in its less approximate estimate of the path of motion as compared to the other two integrators.

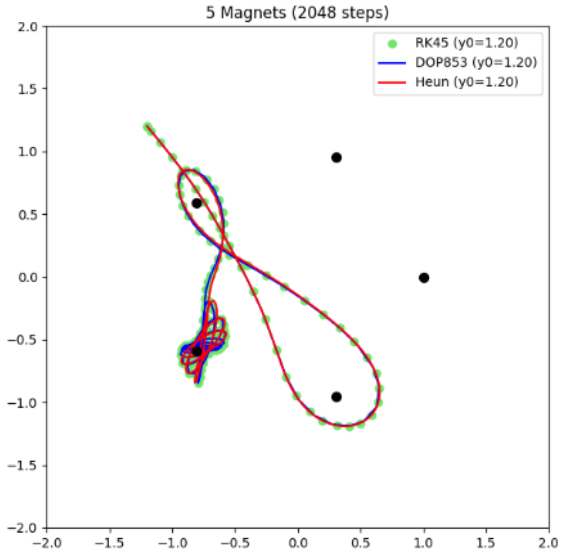

Figure 12: RK45, Heun's Method, and DOP853 integrators comparison for 512 steps and a system of five magnets with the same initial conditions. Despite having a non-adaptive step size, Heun's Method gives a much better approximation of the motion when it has an increased number of steps.

Table 1: Final trajectory separation distances between RK45, DOP853, and Heun integrators for identical initial conditions under varying magnet configurations and step counts. Notice that Heun's Method has less distance between RK45 and DOP853 lines of motion as more steps are allowed. This is consistent with the theory.

Magnets	**Steps**	**RK45 vs DOP853**	**RK45 vs Heun**	**Heun vs DOP853**
3	512	0.000010	1.600657	1.600665
3	2048	0.000010	0.046943	0.046935
4	512	1.369746	0.690705	0.683132
4	2048	1.369746	2.065917	2.069186
5	512	0.000006	1.236992	1.236997
5	2048	0.000006	0.014826	0.014832

4 Lessons Learned

I really enjoyed this project and the making of it. It was nice to incorporate bits and pieces of all the projects together. I used the randomness from Project 4 to show different initial conditions for the magnetic pendulum system, and also the animation function again for some nice GIFs. I used different integration methods like in Project 3 to map the motion and show how it diverges in a more chaotic system. I think this project gave me a better appreciation for the different integration methods and how accuracy works for those.

I attempted to be more linear in my workflow for this project. I set reasonable goals for my daily commits. This reduced stress, improved the clarity of my edits, and overall helped me be more organized in making this. I will also say that condensing the first two sections of code into something that would work for any number of magnets also helped with the understanding of what was going on.

I think the takeaway from this project was mostly excitement! I am looking forward to exploring this project's contents for different pendulums or other chaotic systems over the summer. It was a lot of fun for me to generate fun pictures, which helped in my excitement towards the project.

4.1 Time Keeping

The code took around fifteen hours to generate between outlining the raw structure and fine tuning small details in the plots and animations. There was also a good portion of time trying to streamline the code for visual clarity. The writeup took around seven hours to write and edit. The slides took around three hours to research and put together. All in all, with additional time added for contemplation of the project and its next steps, time spent researching the topic, and talking with classmates about different things to explore within the project scope, I would say the final project took around thirty hours.

4.2 Attributions

- `https://matplotlib.org/stable/api/_as_gen/matplotlib.colors.ListedColormap.html` - A useful guide to using the colormap library.

- https://imamagnets.com/en/blog/magnetic-pendulum/#:~:text=With%20a%20magnetic%20pendulum%2C%20several,a%20certain%20line%20of%20movement – How to construct a magnetic pendulum.
- https://physics.stackexchange.com/questions/638872/deterministic-and-stochastic-cha - Good explanation of deterministic chaos.
- https://beltoforion.de/en/magnetic_pendulum/ - How to simulate the magnetic pendulum.
- https://fractalfoundation.org/OFC/OFC-7-2.html - Gives the equations of motion for the magnetic pendulum.
- https://en.wikipedia.org/wiki/Fractal - What is a fractal, and why is it interesting?
- https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.DOP853.html - Documentation on the DOP853 method.
- https://en.wikipedia.org/wiki/Heun%27s_method - Heun's Method and how it is an improvement on Euler's Method.
- https://en.wikipedia.org/wiki/RungeKuttaFehlberg_method - RK45 method and the math behind it.

- Michael Potts, who suggested the magnetic pendulum as a cool, fractal-generating final project. I did some research and fell in love with the idea, so thank you to him for bringing it to my attention.
- Dr. Reid, who encouraged me to step outside the box and find a topic for this project that I was passionate about.